

```

In [21]: # =====
# 🟦 NAS100 Bot – Moteur de Backtest Tick-by-Tick
# =====
# Architecture : M5 (signaux VWAP/ATR) → Ticks (exécution)
# Instrument : NAS100 / US100 (VTMarkets ECN)
# Cahier des charges : NAS100_Cahier_des_Charges_Quantitatif.md
# Adapté de : Gold Bot v2 (XAUUSD) – recalibration complète
# Données : 1 an | ~141M+ ticks | Polars LazyFrame (Parquet)
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import warnings
import time
import pickle
import multiprocessing
from concurrent.futures import ThreadPoolExecutor, as_completed
from datetime import timedelta
from pathlib import Path
from collections import OrderedDict
import copy

from tqdm.notebook import tqdm

try:
    import polars as pl
    print(f'✅ Polars {pl.__version__} disponible')
except ImportError:
    raise ImportError('Installez polars : pip install polars')

warnings.filterwarnings('ignore')
plt.style.use('dark_background')

# — Chemins de données —————
PATH_M1 = Path('data/NAS100._M1_1Year.csv')
PATH_M5 = Path('data/NAS100._M5_1Year.csv')
PATH_TICKS = Path('data/NAS100._Ticks_1Year.csv')
PATH_TICKS_PARQ = Path('data/NAS100._Ticks_1Year.parquet')
CACHE_INDICATORS = Path('data/nas100._precomp_indicators.pkl')

# — Paramètres contractuels NAS100 —————
CONTRACT_MULTIPLIER_NAS100 = 10.0 # USD/pt/lot standard
LOT_BASE_NAS100 = 0.10
LOT_MIN_NAS100 = 0.01
LOT_MAX_NAS100 = 0.20
# PNL_PER_POINT = LOT_BASE × CONTRACT_MULTIPLIER = 0.10 × 10 = 1.00 $/pt
PNL_PER_POINT = 1.00 # $/pt – NE PAS REDÉFINIR AILLEURS

# — Friction NAS100 —————
SLIPPAGE_PTS = 0.05 # pts – gap Open US (vs 0.5 XAUUSD)

```

```

MAX_SPREAD_NAS100 = 1.5 # pts - refus si spread > 1.5

# — Risk management NAS100 —————
MAX_RISK_PTS      = 60.0 # pts index (vs 22.0 XAUUSD)
DAILY_MAX_LOSS    = -200.0 # $ (vs -150 XAUUSD)
MAX_TRADES_PER_DAY= 20 # (vs 60 XAUUSD)
MAX_CONSECUTIVE_SL= 2 # circuit breaker (vs 3 XAUUSD)
COOLDOWN_SL_MULT  = 2.0 # décroissance agressive (vs 1.5)

# — Fenêtre temporelle NAS100 (GMT+2) —————
SESSION_PREOPEN_H = 14 # actif, seuil +15
SESSION_OPEN_H     = 15 # fenêtre principale, seuil nominal
SESSION_MID_H      = 16 # post-premier-quart-d'heure, seuil +10
SESSION_END_H      = 18 # hard block absolu
GMT_OFFSET         = -1 # Broker GMT+3 → PC GMT+2

# — Paramètres trailing stop NAS100 —————
ATR_TRAIL_P1       = 0.4 # vs 0.5 XAUUSD
TP2_LOCK_RATIO     = 0.25 # vs 0.30 XAUUSD
DELAYED_SL_CANDLES= 1 # 1 bougie M5 (vs 2 XAUUSD)
DELAYED_SL_MULT    = 1.2 # vs 1.0 XAUUSD

# — Paramètres indicateurs NAS100 —————
VWAP_SLOPE_ATR_RATIO = 0.08 # vs 0.10 XAUUSD
SD_CAP               = 4.5
P2_CONFIRM_THRESH_MULT= 0.5 # vs 0.3 XAUUSD
SWING_BUFFER_MULT    = 0.30 # vs 0.20 XAUUSD

# — Modes NAS100 —————
MODES_NAS100 = {
    'safe': {
        'sl_mult': 2.5, 'tp1_mult': 1.2, 'tp2_mult': 2.5, 'tp3_mult':
4.5,
        'score_min': 65, 'cooldown': 360,
    },
    'risque': {
        'sl_mult': 2.0, 'tp1_mult': 1.2, 'tp2_mult': 2.2, 'tp3_mult':
4.0,
        'score_min': 55, 'cooldown': 240,
    },
    'optimale': {
        'sl_mult': 1.8, 'tp1_mult': 1.2, 'tp2_mult': 2.5, 'tp3_mult':
5.0,
        'score_min': 50, 'cooldown': 180,
    },
}

TRADING_MODE = 'optimale'

print('✅ Configuration NAS100 OK')
print(f' Mode : {TRADING_MODE.upper()}')
print(f' Score min : {MODES_NAS100[TRADING_MODE]["score_min"]}')
```

```
print(f'    MAX_RISK      : {MAX_RISK_PTS} pts →  
{MAX_RISK_PTS*PNL_PER_POINT:.0f}$ /sous-trade')  
print(f'    Session      : {SESSION_PREOPEN_H}h-{SESSION_END_H}h GMT+2')
```

✅ Polars 1.40.1 disponible
✅ Configuration NAS100 OK

Mode	: OPTIMALE
Score min	: 50
SL mult	: 1.8
PNL/pt	: 1.0 \$/pt (lot 0.1)
Slippage	: 0.05 pts
MAX_RISK	: 60.0 pts → 60\$/sous-trade
Session	: 14h-18h GMT+2

In [22]:

```
# =====
# CELLULE 2 – Chargement M5 & M1 (pandas) + détection format ticks
# =====

def load_csv(path: Path, label: str) -> pd.DataFrame:
    """Charge CSV OHLCV broker avec correction GMT+3 → GMT+2."""
    df = pd.read_csv(path, parse_dates=['Time'])
    df.rename(columns={'TickVolume': 'Volume'}, inplace=True)
    df['Time'] = df['Time'] + pd.Timedelta(hours=GMT_OFFSET)
    df.set_index('Time', inplace=True)
    df.sort_index(inplace=True)
    df[['Open', 'High', 'Low', 'Close', 'Volume']] = \
        df[['Open', 'High', 'Low', 'Close', 'Volume']].astype(float)
    print(f'✅ {label} : {len(df):,} bougies | {df.index[0]} → {df.index[-1]}')
    return df

df_m5 = load_csv(PATH_M5, 'NAS100 M5')
df_m1 = load_csv(PATH_M1, 'NAS100 M1')

# — Détection format ticks —————
print('\n🔍 Format tick CSV :')
with open(PATH_TICKS, 'r') as f:
    for i, line in enumerate(f):
        print(f'  Ligne {i}: {line.strip()}')
        if i >= 3:
            break

tick_size_gb = PATH_TICKS.stat().st_size / 1e9
print(f'\n📁 Fichier ticks : {tick_size_gb:.2f} GB')
print('  → Polars LazyFrame Parquet : filtrage par trade, jamais tout en RAM')
```

```
✅ NAS100 M5 : 70,790 bougies | 2025-05-09 00:00:00 → 2026-05-08 22:55:00
✅ NAS100 M1 : 353,899 bougies | 2025-05-09 00:00:00 → 2026-05-08 22:59:00
```

```
🔍 Format tick CSV :
  Ligne 0: Time,Bid,Ask,Last,Volume
  Ligne 1: 2025.05.09 01:00:08.632,20089.55,20093.50,0.00,0
  Ligne 2: 2025.05.09 01:00:09.578,20089.05,20093.00,0.00,0
  Ligne 3: 2025.05.09 01:00:10.165,20090.30,20094.00,0.00,0
```

```
📁 Fichier ticks : 2.46 GB
  → Polars LazyFrame Parquet : filtrage par trade, jamais tout en RAM
```

In [23]:

```

# =====
# CELLULE 2b – Conversion CSV → Parquet (UNE SEULE FOIS)
# =====
# Crée data/NAS100_Ticks_1Year.parquet (~1-2 GB).
# Après première exécution : le bloc est ignoré (fichier existant).
# =====

def convert_ticks_to_parquet():
    if PATH_TICKS_PARQ.exists():
        size_gb = PATH_TICKS_PARQ.stat().st_size / 1e9
        print(f'✅ Parquet déjà existant ({size_gb:.2f} GB) –
conversion ignorée')
        return

    print(f'⌚ Conversion CSV → Parquet
({PATH_TICKS.stat().st_size/1e9:.1f} GB)...')
    print('  Durée estimée : 5-15 min (SSD ~5 min, HDD ~15 min)')
    t0 = time.time()

    # Détection colonnes
    with open(PATH_TICKS, 'r') as f:
        raw_cols = [c.strip().strip('<>') for c in
f.readline().strip().split(',')]

    time_col = next((c for c in raw_cols if 'time' in c.lower()),
raw_cols[0])
    bid_col = next((c for c in raw_cols if 'bid' in c.lower()), None)
    ask_col = next((c for c in raw_cols if 'ask' in c.lower()), None)

    if bid_col is None or ask_col is None:
        raise ValueError(f'Colonnes Bid/Ask non trouvées dans :
{raw_cols}')

    print(f'  Colonnes : Time="{time_col}" Bid="{bid_col}" Ask="
{ask_col}"')

    (
        pl.scan_csv(PATH_TICKS, try_parse_dates=False,
infer_schema_length=500)
        .select([time_col, bid_col, ask_col])
        .rename({time_col: 'RawTime', bid_col: 'Bid', ask_col: 'Ask'})
        .with_columns([
            pl.col('Bid').cast(pl.Float64),
            pl.col('Ask').cast(pl.Float64),
            pl.col('RawTime')
                .str.to_datetime(format='%Y.%m.%d %H:%M:%S%.3f',
strict=False)
                .alias('Time'),
        ])
        .drop('RawTime')
        .sink_parquet(PATH_TICKS_PARQ, compression='snappy')
    )

    elapsed = time.time() - t0
    size_gb = PATH_TICKS_PARQ.stat().st_size / 1e9
    print(f'\n✅ Parquet créé : {size_gb:.2f} GB en {elapsed:.0f}s')
    print(f'  (Lectures futures : ~50x plus rapides)')

```

```
convert_ticks_to_parquet()

# LazyFrame global (réutilisé dans toutes les requêtes tick)
_TICKS_LAZY = pl.scan_parquet(PATH_TICKS_PARQ)
print('\n✅ LazyFrame Parquet NAS100 prêt')
```

✅ Parquet déjà existant (0.42 GB) – conversion ignorée

✅ LazyFrame Parquet NAS100 prêt

In [24]:

```

# =====
# CELLULE 3 – Fonctions Indicateurs (répliques bot_headless NAS100)
# =====

def compute_vwap(df: pd.DataFrame, window: int = 50) -> pd.Series:
    typical = (df['High'] + df['Low'] + df['Close']) / 3
    tp_vol = typical * df['Volume']
    return tp_vol.rolling(window).sum() /
df['Volume'].rolling(window).sum()

def compute_volume_profile(df: pd.DataFrame, bins: int = 20) -> dict:
    price_min, price_max = df['Low'].min(), df['High'].max()
    if price_max == price_min:
        return {'poc': df['Close'].iloc[-1], 'value_area_high':
price_max,
                'value_area_low': price_min}
    volumes, edges = np.histogram(df['Close'], bins=bins,
weights=df['Volume'])
    poc_idx = volumes.argmax()
    poc = (edges[poc_idx] + edges[poc_idx + 1]) / 2
    target = volumes.sum() * 0.70
    sorted_i = np.argsort(volumes)[::-1]
    acc, va_i = 0, []
    for idx in sorted_i:
        acc += volumes[idx]; va_i.append(idx)
        if acc >= target:
            break
    va_high = max((edges[i] + edges[i+1]) / 2 for i in va_i)
    va_low = min((edges[i] + edges[i+1]) / 2 for i in va_i)
    return {'poc': round(poc, 2), 'value_area_high': round(va_high, 2),
            'value_area_low': round(va_low, 2)}

def compute_atr(df: pd.DataFrame, window: int = 14) -> float:
    tr = pd.concat([
        df['High'] - df['Low'],
        (df['High'] - df['Close'].shift()).abs(),
        (df['Low'] - df['Close'].shift()).abs(),
    ], axis=1).max(axis=1)
    return round(tr.rolling(window).mean().iloc[-1], 2)

def _detect_candle_pattern(df: pd.DataFrame) -> str:
    o, h, l, c = df['Open'].iloc[-1], df['High'].iloc[-1], \
        df['Low'].iloc[-1], df['Close'].iloc[-1]
    body = abs(c - o)
    candle = h - l
    if candle == 0:
        return 'Doji'
    ratio = body / candle
    lower_wick = min(o, c) - l
    upper_wick = h - max(o, c)
    if ratio < 0.1:
        return
'Doji'
    if lower_wick > body*2 and upper_wick < body*0.5 and c > o: return
'Marteau'

```

```

    if upper_wick > body*2 and lower_wick < body*0.5 and c < o: return
'Shooting Star'
    prev_o, prev_c = df['Open'].iloc[-2], df['Close'].iloc[-2]
    if c > o and c > prev_o and o < prev_c and prev_c < prev_o: return
'Engulfing haussier'
    if c < o and c < prev_o and o > prev_c and prev_c > prev_o: return
'Engulfing baissier'
    if lower_wick > body * 3:
return
'Pin Bar haussier'
    if upper_wick > body * 3:
return
'Pin Bar baissier'
    return 'Neutre'

```

```

def compute_indicators(df: pd.DataFrame) -> dict:
    close, high, low, vol = df['Close'], df['High'], df['Low'],
df['Volume']
    price = close.iloc[-1]
    atr = compute_atr(df)

    vwap_series = compute_vwap(df, window=50)
    vwap = vwap_series.iloc[-1]
    typical = (high + low + close) / 3
    vwap_std = (typical - vwap_series).rolling(50).std().iloc[-1]
    if np.isnan(vwap_std) or vwap_std == 0:
        vwap_std = 1e-9

    vwap_upper1 = round(vwap + 1.0 * vwap_std, 2)
    vwap_lower1 = round(vwap - 1.0 * vwap_std, 2)
    vwap_upper2 = round(vwap + 2.0 * vwap_std, 2)
    vwap_lower2 = round(vwap - 2.0 * vwap_std, 2)
    dist_vwap = round(price - vwap, 2)
    dist_vwap_sd = round(dist_vwap / vwap_std, 2)
    # NAS100 : slope sur 5 bougies – filtre 0.08×ATR (vs 0.10)
    vwap_slope = round(vwap_series.diff(5).iloc[-1], 2)

    vp = compute_volume_profile(df.tail(100))
    poc = vp['poc']
    near_poc = abs(price - poc) < atr * 0.5

    swing_high = round(df.tail(30)['High'].max(), 2)
    swing_low = round(df.tail(30)['Low'].min(), 2)
    # NAS100 : buffer swing 1.0×ATR (identique XAUUSD)
    near_swing_support = abs(price - swing_low) < atr * 1.0
    near_swing_resistance = abs(price - swing_high) < atr * 1.0

    ema200 = close.ewm(span=200, adjust=False).mean().iloc[-1]
    ema20_s = close.ewm(span=20, adjust=False).mean()
    ema50_s = close.ewm(span=50, adjust=False).mean()
    above_200 = price > ema200
    above_50 = price > ema50_s.iloc[-1]
    ema_cross_bull= (ema20_s.iloc[-1] > ema50_s.iloc[-1] and
ema20_s.iloc[-2] <= ema50_s.iloc[-2])
    ema_cross_bear= (ema20_s.iloc[-1] < ema50_s.iloc[-1] and
ema20_s.iloc[-2] >= ema50_s.iloc[-2])

    avg_vol = vol.rolling(20).mean().iloc[-1]

```



```

vol_ratio = round(vol.iloc[-1] / avg_vol, 2) if avg_vol > 0 else
1.0
vol_spike = vol_ratio > 1.4

cur_open = df['Open'].iloc[-1]; cur_close = close.iloc[-1]
cur_body = abs(cur_close - cur_open)
cur_bull = cur_close > cur_open; cur_bear = cur_close < cur_open
move10 = close.iloc[-11:-1].iloc[-1] -
close.iloc[-11:-1].iloc[0]

# NAS100 bounce – même logique XAUUSD, seuils identiques
bounce_bull = (near_swing_support and cur_bull and cur_body > atr *
0.5
                and vol_ratio > 1.4 and move10 < -atr * 1.5
                and (not above_200 or (above_200 and near_poc)))
bounce_bear = (near_swing_resistance and cur_bear and cur_body > atr
* 0.5
                and vol_ratio > 1.4 and move10 > atr * 1.5)

pattern = _detect_candle_pattern(df)

# Spread proxy via tick data (si dispo) – sinon None (filtré à
l'exécution)
spread_proxy = None # sera rempli par le simulateur tick

return {
    'price': round(price, 2), 'atr': atr,
    'vwap': round(vwap, 2),
    'vwap_upper1': vwap_upper1, 'vwap_lower1': vwap_lower1,
    'vwap_upper2': vwap_upper2, 'vwap_lower2': vwap_lower2,
    'dist_vwap_sd': dist_vwap_sd, 'vwap_slope': vwap_slope,
    'poc': poc, 'va_high': vp['value_area_high'], 'va_low':
vp['value_area_low'],
    'near_poc': near_poc, 'swing_high': swing_high, 'swing_low':
swing_low,
    'near_swing_sup': near_swing_support, 'near_swing_res':
near_swing_resistance,
    'ema200': round(ema200, 2), 'ema20': round(ema20_s.iloc[-1], 2),
    'ema50': round(ema50_s.iloc[-1], 2),
    'above_200': above_200, 'above_50': above_50,
    'ema_cross_bull': ema_cross_bull, 'ema_cross_bear':
ema_cross_bear,
    'vol_ratio': vol_ratio, 'vol_spike': vol_spike,
    'bounce_bull': bounce_bull, 'bounce_bear': bounce_bear,
    'move10': round(move10, 2), 'pattern': pattern,
}

print('✅ Fonctions indicateurs NAS100 définies')

```

✅ Fonctions indicateurs NAS100 définies

In [25]:

```

# =====
# CELLULE 4 – Direction, Score & Niveaux TP/SL (NAS100)
# =====

def determine_direction(ind: dict):
    """Identique XAUUSD avec filtre slope assoupli : 0.08×ATR (vs
    0.10)."""
    # NAS100 : VWAP_SLOPE_ATR_RATIO = 0.08
    if abs(ind['vwap_slope']) < ind['atr'] * VWAP_SLOPE_ATR_RATIO:
        return None, 'vwap_flat'
    dist = ind['dist_vwap_sd']
    if dist > SD_CAP or dist < -SD_CAP:
        return None, 'sd_cap'

    price = ind['price']
    above_200 = ind['above_200']
    above_50 = ind['above_50']

    if ind['bounce_bull']: return 'BUY', 'bounce_bull'
    if ind['bounce_bear']: return 'SELL', 'bounce_bear'

    vwap_long = price < ind['vwap_lower1']
    vwap_short = price > ind['vwap_upper1']
    struct_long = ind['near_swing_sup'] or ind['near_poc']
    struct_short = ind['near_swing_res'] or ind['near_poc']
    vol_ok = ind['vol_ratio'] > 1.0

    # NAS100 : biais BUY dominant (above_200 vrai 80-90% en haussier)
    if (vwap_long and struct_long and above_200 and above_50
        and (vol_ok or ind['ema_cross_bull'])):
        return 'BUY', 'vwap_1sd'
    if (vwap_short and struct_short and not above_200 and not above_50
        and (vol_ok or ind['ema_cross_bear'])):
        return 'SELL', 'vwap_1sd'
    if price < ind['vwap_lower2'] and above_200 and vol_ok:
        return 'BUY', 'vwap_2sd'
    if price > ind['vwap_upper2'] and not above_200 and vol_ok:
        return 'SELL', 'vwap_2sd'
    return None, 'no_setup'

def compute_score(ind: dict, direction: str):
    """Barème identique XAUUSD – ratio-invariant (§7.1 cahier des
    charges)."""
    score = 0; reasons = []; is_buy = direction == 'BUY'
    dist_sd = abs(ind['dist_vwap_sd'])

    if dist_sd >= 2.0: score += 30; reasons.append(f'SD=
    {dist_sd:.1f}>=2.0 (+30)')
    elif dist_sd >= 1.0: score += 18; reasons.append(f'SD=
    {dist_sd:.1f}>=1.0 (+18)')
    elif dist_sd >= 0.5: score += 8; reasons.append(f'SD=
    {dist_sd:.1f}>=0.5 (+8)')

    if is_buy and ind['vwap_slope'] > 0: score += 5;
    reasons.append('Slope+ (+5)')
    elif not is_buy and ind['vwap_slope'] < 0: score += 5;

```

```

reasons.append('Slope- (+5)')

    if is_buy:
        if ind['near_swing_sup']: score += 15; reasons.append('SwingSup
(+15)')
        if ind['near_poc']: score += 10; reasons.append('POC
(+10)')
        if ind['price'] < ind['va_low']: score += 10;
reasons.append('BelowVAL (+10)')
        if ind['above_200']: score += 5; reasons.append('Above200
(+5)')
    else:
        if ind['near_swing_res']: score += 15; reasons.append('SwingRes
(+15)')
        if ind['near_poc']: score += 10; reasons.append('POC
(+10)')
        if ind['price'] > ind['va_high']: score += 10;
reasons.append('AboveVAH (+10)')
        if not ind['above_200']: score += 5; reasons.append('Below200
(+5)')

    vr = ind['vol_ratio']
    if ind['bounce_bull'] or ind['bounce_bear']: score += 25;
reasons.append('Bounce (+25)')
    elif vr >= 2.0: score += 20; reasons.append(f'VolSpike x{vr} (+20)')
    elif vr >= 1.4: score += 12; reasons.append(f'VolHigh x{vr} (+12)')
    elif vr >= 1.0: score += 5; reasons.append(f'VolOK x{vr} (+5)')

    if is_buy and ind['ema_cross_bull']: score += 10;
reasons.append('GoldenX (+10)')
    elif not is_buy and ind['ema_cross_bear']: score += 10;
reasons.append('DeathX (+10)')

    bull_p = {'Marteau', 'Engulfing haussier', 'Pin Bar haussier'}
    bear_p = {'Shooting Star', 'Engulfing baissier', 'Pin Bar baissier'}
    if is_buy and ind['pattern'] in bull_p: score += 15;
reasons.append(f"{ind['pattern']} (+15)")
    elif not is_buy and ind['pattern'] in bear_p: score += 15;
reasons.append(f"{ind['pattern']} (+15)")
    elif ind['pattern'] == 'Doji': score += 3; reasons.append('Doji
(+3)')

    return min(score, 100), reasons

def calculate_tp_sl(direction: str, entry: float, atr: float,
                    swing_high: float, swing_low: float,
                    mode: str = TRADING_MODE) -> dict:
    """
    NAS100 : buffer swing 0.30×ATR (vs 0.20 XAUUSD), cap 60 pts.
    """
    m = MODES_NAS100[mode]
    MXRK = MAX_RISK_PTS
    if direction == 'BUY':
        sl = min(round(swing_low - atr * SWING_BUFFER_MULT, 2),
                  round(entry - atr * m['sl_mult'], 2))
        sl = min(sl, round(entry - atr * max(m['sl_mult'], 1.5), 2))

```

```

risk = abs(entry - sl)
if risk > MXRK:
    sl = round(entry - MXRK, 2); risk = MXRK
return {'sl': sl,
        'tp1': round(entry + risk * m['tp1_mult'], 2),
        'tp2': round(entry + risk * m['tp2_mult'], 2),
        'tp3': round(entry + risk * m['tp3_mult'], 2)}
else:
    sl = max(round(swing_high + atr * SWING_BUFFER_MULT, 2),
              round(entry + atr * m['sl_mult'], 2))
    sl = max(sl, round(entry + atr * max(m['sl_mult'], 1.5), 2))
    risk = abs(sl - entry)
    if risk > MXRK:
        sl = round(entry + MXRK, 2); risk = MXRK
    return {'sl': sl,
            'tp1': round(entry - risk * m['tp1_mult'], 2),
            'tp2': round(entry - risk * m['tp2_mult'], 2),
            'tp3': round(entry - risk * m['tp3_mult'], 2)}

def compute_lot_dynamic(score: int, atr: float, hour: int,
                        median_atr_30d: float) -> float:
    """
    §8 Sizing dynamique NAS100.
    lot_scale score≥80 : x1.3 (vs x1.5 XAUUSD)
    lot_scale ATR spike : x0.6 (vs x0.7 XAUUSD)
    """
    multiplier = 1.0
    if score >= 80: multiplier *= 1.3
    elif score <= 55: multiplier *= 0.7
    if median_atr_30d > 0 and atr > 1.5 * median_atr_30d:
        multiplier *= 0.6
    if hour >= SESSION_MID_H:
        multiplier *= 0.8
    lot = round(LOT_BASE_NAS100 * multiplier, 2)
    return max(LOT_MIN_NAS100, min(LOT_MAX_NAS100, lot))

print('✅ Fonctions signal, score, niveaux et sizing NAS100 définies')

```

✅ Fonctions signal, score, niveaux et sizing NAS100 définies

In [26]:

```

# =====
# CELLULE 5a – Simulateur M1 NAS100 (référence de comparaison)
# =====
# Trailing stop NAS100 :
#   Post-TP1 : SL → entry + 0.4×ATR (vs 0.5 XAUUSD)
#   Post-TP2 : SL → tp1 + 0.25×(tp2-tp1) (vs 0.30 XAUUSD)
# =====

def simulate_trade_m1(direction: str, entry: float, levels: dict,
                      open_time: pd.Timestamp, df_m1: pd.DataFrame,
                      atr: float = 0.0) -> list:
    """
    Simulateur M1 NAS100 avec trailing stop ATR (§5 cahier des charges).
    Inclut : delayed SL après 1 bougie M5, trailing post-TP1 et post-
    TP2.
    """
    sl_orig = levels['sl']
    tp1, tp2, tp3 = levels['tp1'], levels['tp2'], levels['tp3']

    start_pos = df_m1.index.searchsorted(open_time)
    bars = df_m1.iloc[start_pos:]
    if bars.empty:
        return []

    positions = [
        {'id': 'P1', 'tp': tp1, 'sl': sl_orig, 'result': None,
         'exit_px': None, 'pn1': 0.0, 'closed': False},
        {'id': 'P2', 'tp': tp2, 'sl': sl_orig, 'result': None,
         'exit_px': None, 'pn1': 0.0, 'closed': False},
        {'id': 'P3', 'tp': tp3, 'sl': sl_orig, 'result': None,
         'exit_px': None, 'pn1': 0.0, 'closed': False},
    ]

    tp1_hit = tp2_hit = False
    highs = bars['High'].values
    lows = bars['Low'].values
    closes = bars['Close'].values

    # Delayed SL après DELAYED_SL_CANDLES bougies M5 = 1×5 = 5 barres M1
    DELAYED_BARS_M1 = DELAYED_SL_CANDLES * 5
    delayed_applied = False

    for k in range(len(highs)):
        if all(p['closed'] for p in positions):
            break

        # — Delayed SL resserrement (§3.4) —————
        if not delayed_applied and k >= DELAYED_BARS_M1 and atr > 0:
            delayed_applied = True
            if direction == 'BUY':
                new_sl = round(entry - DELAYED_SL_MULT * atr, 2)
            else:
                new_sl = round(entry + DELAYED_SL_MULT * atr, 2)
            for p in positions:
                if not p['closed']:
                    # Appliquer seulement si resserre
                    if direction == 'BUY' and new_sl > p['sl']:
                        p['sl'] = new_sl

```

```

elif direction == 'SELL' and new_sl < p['sl']:
    p['sl'] = new_sl

bh, bl = highs[k], lows[k]
for p in positions:
    if p['closed']:
        continue
    tp_hit = (bh >= p['tp']) if direction == 'BUY' else (bl <=
p['tp'])
    sl_hit = (bl <= p['sl']) if direction == 'BUY' else (bh >=
p['sl'])

    if tp_hit and sl_hit:
        tp_hit = abs(p['tp'] - entry) < abs(p['sl'] - entry)
        sl_hit = not tp_hit

    if tp_hit:
        p['closed'] = True; p['exit_px'] = p['tp']
        pts = (p['tp'] - entry) if direction == 'BUY' else
(entry - p['tp'])
        p['result'] = p['id'].replace('P', 'TP')
        p['pn1'] = round(pts * PNL_PER_POINT, 2)

    if p['id'] == 'P1' and not tp1_hit:
        tp1_hit = True
        # Post-TP1 trailing : SL → entry + 0.4×ATR ($5.1)
        if atr > 0:
            trail_sl = round((entry + ATR_TRAIL_P1 * atr)
                             if direction == 'BUY'
                             else (entry - ATR_TRAIL_P1 *
atr), 2)

        else:
            trail_sl = entry
        for o in positions:
            if not o['closed']:
                if direction == 'BUY' and trail_sl >
o['sl']:
                    o['sl'] = trail_sl
                elif direction == 'SELL' and trail_sl <
o['sl']:
                    o['sl'] = trail_sl

    if p['id'] == 'P2' and not tp2_hit and tp1_hit:
        tp2_hit = True
        # Post-TP2 lock : SL → tp1 + 0.25×(tp2-tp1) ($5.2)
        if direction == 'BUY':
            lock_sl = round(tp1 + TP2_LOCK_RATIO * (tp2 -
tp1), 2)

        else:
            lock_sl = round(tp1 - TP2_LOCK_RATIO * (tp1 -
tp2), 2)

        for o in positions:
            if not o['closed']:
                if direction == 'BUY' and lock_sl >
o['sl']:
                    o['sl'] = lock_sl
                elif direction == 'SELL' and lock_sl <

```

```

o['sl']:

                                o['sl'] = lock_sl

        elif sl_hit:
            p['closed'] = True; p['exit_px'] = p['sl']
            pts = abs(entry - p['sl'])
            p['result'] = 'BE' if abs(p['sl'] - entry) < 0.01 else
'SL'
            p['pnl']      = 0.0 if p['result'] == 'BE' else round(-pts
* PNL_PER_POINT, 2)

        last_close = closes[-1]
        for p in positions:
            if not p['closed']:
                pts = (last_close - entry) if direction == 'BUY' else (entry
- last_close)
                p['result']  = 'OPEN_END'
                p['exit_px'] = last_close
                p['pnl']      = round(pts * PNL_PER_POINT, 2)
                p['closed']  = True

        return positions

print('✅ Simulateur M1 NAS100 (avec trailing ATR) défini')

```

✅ Simulateur M1 NAS100 (avec trailing ATR) défini

In [27]:

```

# =====
# CELLULE 5b – Fetch Ticks + Simulateur Tick NAS100 (Numpy pur)
# =====
# CORRECTION Time-Travel : exec_time = ts + 5 min (clôture bougie M5)
# CORRECTION friction : SLIPPAGE_PTS injecté explicitement
# NAS100 specifics :
# - Filtre spread MAX_SPREAD_NAS100 = 1.5 pts
# - Trailing post-TP1 : 0.4×ATR, post-TP2 : 0.25×(tp2-tp1)
# - Delayed SL : après 1 bougie M5 (= 60 ticks M1 env.)
# =====

def _fetch_ticks_numpy(signal_time_pc: pd.Timestamp,
                      max_hours: int = 48) -> tuple:
    """
    Extrait ticks depuis Parquet.
    signal_time_pc : timestamp GMT+2 APRÈS décalage +5min (clôture M5).
    Retourne (bids, asks) en numpy, ou ([], []) si vide.
    """
    broker_start = signal_time_pc + timedelta(hours=-GMT_OFFSET)
    broker_end = broker_start + timedelta(hours=max_hours)

    df = (
        _TICKS_LAZY
        .filter(
            (pl.col('Time') >= broker_start) &
            (pl.col('Time') <= broker_end)
        )
        .select(['Bid', 'Ask'])
        .collect(streaming=True)
    )
    if df.is_empty():
        return np.array([]), np.array([])
    return df['Bid'].to_numpy(), df['Ask'].to_numpy()

def simulate_trade_ticks_fast(direction: str,
                              entry_signal: float,
                              levels: dict,
                              open_time,
                              bids: np.ndarray,
                              asks: np.ndarray,
                              atr: float = 0.0,
                              slippage: float = SLIPPAGE_PTS) -> list:
    """
    Simulateur tick NAS100.
    - Spread check au premier tick (refus si spread > MAX_SPREAD_NAS100)
    - Trailing post-TP1 (0.4×ATR) et post-TP2 (0.25×range)
    - Delayed SL après 5 minutes (≈ premier tick de la 2ème bougie M5)
    - MFE/MAE tracking pour analyse analytique
    """
    if len(bids) == 0:
        return []

    # — Filtre spread à l'entrée (§2.2) —————
    spread_0 = asks[0] - bids[0]
    if spread_0 > MAX_SPREAD_NAS100:
        return [] # signal rejeté – spread trop large

```



```

# — Prix d'entrée réel avec slippage —————
entry = round((asks[0] + slippage) if direction == 'BUY'
               else (bids[0] - slippage), 2)
delta = entry - entry_signal

sl_init = round(levels['sl'] + delta, 2)
tp1      = round(levels['tp1'] + delta, 2)
tp2      = round(levels['tp2'] + delta, 2)
tp3      = round(levels['tp3'] + delta, 2)

positions = [
    {'id': 'P1', 'tp': tp1, 'sl': sl_init, 'result': None,
'exit_px': None,
    'pnl': 0.0, 'closed': False, 'entry_real': entry,
    'slippage': round(abs(delta), 4), 'mfe': 0.0, 'mae': 0.0},
    {'id': 'P2', 'tp': tp2, 'sl': sl_init, 'result': None,
'exit_px': None,
    'pnl': 0.0, 'closed': False, 'entry_real': entry,
    'slippage': round(abs(delta), 4), 'mfe': 0.0, 'mae': 0.0},
    {'id': 'P3', 'tp': tp3, 'sl': sl_init, 'result': None,
'exit_px': None,
    'pnl': 0.0, 'closed': False, 'entry_real': entry,
    'slippage': round(abs(delta), 4), 'mfe': 0.0, 'mae': 0.0},
]
tp1_hit = tp2_hit = delayed_applied = False

# Seuil Delayed SL : après ~60 ticks (1 bougie M5 ≈ 60 ticks à 5
ticks/s)
# On utilise un compteur de ticks comme proxy temporel
DELAYED_TICK_THRESHOLD = 60 # ajustable selon densité tick

exec_prices = bids if direction == 'BUY' else asks

for k in range(len(exec_prices)):
    if all(p['closed'] for p in positions):
        break

    px = exec_prices[k]

# — MFE / MAE tracking —————
for p in positions:
    if not p['closed']:
        excursion = (px - entry) if direction == 'BUY' else
(entry - px)
        p['mfe'] = max(p['mfe'], excursion)
        p['mae'] = min(p['mae'], excursion) # négatif =
adverse

# — Delayed SL (§3.4) —————
if not delayed_applied and k >= DELAYED_TICK_THRESHOLD and atr >
0:
    delayed_applied = True
    if direction == 'BUY':
        new_sl = round(entry - DELAYED_SL_MULT * atr, 2)
    else:
        new_sl = round(entry + DELAYED_SL_MULT * atr, 2)

```

```

for p in positions:
    if not p['closed']:
        if direction == 'BUY' and new_sl > p['sl']:
            p['sl'] = new_sl
        elif direction == 'SELL' and new_sl < p['sl']:
            p['sl'] = new_sl

# — Logique TP/SL —————
for p in positions:
    if p['closed']:
        continue
    tp_hit = (px >= p['tp']) if direction == 'BUY' else (px <=
p['tp'])
    sl_hit = (px <= p['sl']) if direction == 'BUY' else (px >=
p['sl'])

    if tp_hit and sl_hit:
        tp_hit = abs(p['tp'] - entry) < abs(p['sl'] - entry)
        sl_hit = not tp_hit

    if tp_hit:
        p['closed'] = True; p['exit_px'] = p['tp']
        pts = (p['tp'] - entry) if direction == 'BUY' else
(entry - p['tp'])
        p['result'] = p['id'].replace('P', 'TP')
        p['pn1'] = round(pts * PNL_PER_POINT, 2)

        if p['id'] == 'P1' and not tp1_hit:
            tp1_hit = True
            if atr > 0:
                trail_sl = round((entry + ATR_TRAIL_P1 * atr)
                                if direction == 'BUY'
                                else (entry - ATR_TRAIL_P1 *
atr), 2)

            else:
                trail_sl = entry
        for o in positions:
            if not o['closed']:
                if direction == 'BUY' and trail_sl >
o['sl']:
                    o['sl'] = trail_sl
                elif direction == 'SELL' and trail_sl <
o['sl']:
                    o['sl'] = trail_sl

        if p['id'] == 'P2' and not tp2_hit and tp1_hit:
            tp2_hit = True
            if direction == 'BUY':
                lock_sl = round(tp1 + TP2_LOCK_RATIO * (tp2 -
tp1), 2)
            else:
                lock_sl = round(tp1 - TP2_LOCK_RATIO * (tp1 -
tp2), 2)

            for o in positions:
                if not o['closed']:
                    if direction == 'BUY' and lock_sl >
o['sl']:

```

```

        o['sl'] = lock_sl
        elif direction == 'SELL' and lock_sl <
o['sl']:
        o['sl'] = lock_sl

        elif sl_hit:
            p['closed'] = True; p['exit_px'] = p['sl']
            pts = abs(entry - p['sl'])
            p['result'] = 'BE' if abs(p['sl'] - entry) < 0.01 else
'SL'
            p['pnl'] = 0.0 if p['result'] == 'BE' \
                else round(-pts * PNL_PER_POINT, 2)

        last_px = exec_prices[-1]
        for p in positions:
            if not p['closed']:
                pts = (last_px - entry) if direction == 'BUY' else (entry -
last_px)
                p['result'] = 'OPEN_END'
                p['exit_px'] = last_px
                p['pnl'] = round(pts * PNL_PER_POINT, 2)
                p['closed'] = True

        return positions

print('✅ Simulateur tick NAS100 (spread-check + trailing ATR +
MFE/MAE) défini')
```

✅ Simulateur tick NAS100 (spread-check + trailing ATR + MFE/MAE) défini

In [28]:

```

# =====
# CELLULE 6 – Pré-calcul des indicateurs M5 (avec cache disque)
# =====

def precompute_all_indicators(df_m5: pd.DataFrame,
                             window: int = 100) -> OrderedDict:
    if CACHE_INDICATORS.exists():
        print('📁 Cache disque détecté – chargement...')
        with open(CACHE_INDICATORS, 'rb') as f:
            result = pickle.load(f)
        print(f'✅ {len(result):,} timestamps chargés depuis cache')
        return result

    t0 = time.time()
    times = df_m5.index.tolist()
    precomp= OrderedDict()
    total = len(df_m5)

    with tqdm(total=total - window,
              desc='Pré-calcul indicateurs M5 NAS100',
              unit='bougies', ncols=80) as pbar:
        for i in range(window, total):
            ts = times[i]
            try:
                precomp[ts] = compute_indicators(df_m5.iloc[i - window:
i + 1])
            except Exception:
                pass
            pbar.update(1)

    elapsed = time.time() - t0
    print(f'\n✅ Pré-calcul terminé : {len(precomp):,} timestamps en
{elapsed:.1f}s')

    with open(CACHE_INDICATORS, 'wb') as f:
        pickle.dump(precomp, f)
    print(f'  Cache sauvegardé → {CACHE_INDICATORS}')

    return precomp

if 'precomp_indicators' in globals() and precomp_indicators:
    print('✅ Indicateurs déjà en RAM – calcul ignoré.')
else:
    print('⌚ Pré-calcul indicateurs NAS100...')
    precomp_indicators = precompute_all_indicators(df_m5)

```

✅ Indicateurs déjà en RAM – calcul ignoré.

In [29]:

```

# =====
# CELLULE 6b – Pré-cache des fenêtres de ticks (correction Time-Travel)
# =====
# CORRECTION : exec_time = ts + 5min (clôture M5), pas ts (ouverture).
# NAS100 : session filtrée [SESSION_PREOPEN_H, SESSION_END_H[
# =====

def _collect_signal_times(precomp: OrderedDict,
                          mode: str,
                          min_score: int = 35) -> list:
    """Collecte timestamps candidats pour le cache tick."""
    m = MODES_NAS100[mode].copy()
    m['score_min'] = min_score

    signal_times = []
    last_signal_ts = pd.Timestamp('1970-01-01')
    daily_pnl = 0.0
    current_date = None
    daily_stopped = False
    trades_today = 0
    consec_sl = 0

    for ts, ind in precomp.items():
        day = ts.date()
        if day != current_date:
            current_date = day
            daily_pnl = 0.0; trades_today = 0
            daily_stopped = False; consec_sl = 0

        h = ts.hour
        if daily_stopped or h < SESSION_PREOPEN_H or h >= SESSION_END_H:
            continue
        if trades_today >= MAX_TRADES_PER_DAY:
            continue
        if (ts - last_signal_ts).total_seconds() < m['cooldown']:
            continue

        direction, _ = determine_direction(ind)
        if direction is None:
            continue
        score, _ = compute_score(ind, direction)
        if score < min_score:
            continue

        signal_times.append(ts)
        last_signal_ts = ts
        trades_today += 1

    return signal_times

def build_tick_cache(signal_times: list, max_hours: int = 48) -> dict:
    """Cache RAM tick par signal. Correction Time-Travel incluse."""
    cache = {}
    print(f'📦 Chargement de {len(signal_times)} fenêtres de ticks...')
    t0 = time.time()

```

```

for ts in tqdm(signal_times, desc='Cache ticks NAS100',
               unit='trade', ncols=70):
    # ✅ CORRECTION : clôture de la bougie M5 = ts + 5 min
    exec_time = ts + pd.Timedelta(minutes=5)
    bids, asks = _fetch_ticks_numpy(exec_time, max_hours=max_hours)
    if len(bids) > 0:
        cache[ts] = (bids, asks)
    else:
        print(f' ⚠️ Aucun tick pour {ts} (exec={exec_time})')

elapsed = time.time() - t0
total_mb = sum(b.nbytes + a.nbytes for b, a in cache.values()) / 1e6
print(f'\n✅ Cache prêt : {len(cache)} trades | {total_mb:.1f} MB
RAM | {elapsed:.0f}s')
return cache

print('🔍 Collecte des timestamps candidats (score_min=35)...')
all_signal_times = _collect_signal_times(precomp_indicators,
                                         TRADING_MODE, min_score=35)
print(f' → {len(all_signal_times)} signaux candidats\n')

print('⌚ Construction du cache ticks...')
TICK_CACHE = build_tick_cache(all_signal_times)

```

🔍 Collecte des timestamps candidats (score_min=35)...
→ 37 signaux candidats

⌚ Construction du cache ticks...

📦 Chargement de 37 fenêtres de ticks...

Cache ticks NAS100: 0%| | 0/37 [00:00<?, ?
trade/s]

✅ Cache prêt : 37 trades | 169.2 MB RAM | 1s

In [30]:

```
# =====
# CELLULE 7 – Pré-calcul ATR médiane 30j (pour sizing dynamique)
# =====

def compute_median_atr_30d(precomp: OrderedDict) -> pd.Series:
    """
    Calcule la médiane glissante 30j de l'ATR M5.
    Utilisée par compute_lot_dynamic (§8).
    """
    records = [(ts, ind['atr']) for ts, ind in precomp.items()]
    s = pd.Series(
        [r[1] for r in records],
        index=pd.DatetimeIndex([r[0] for r in records]),
    )
    # Médiane sur 30 jours glissants (≈ 30×24×12 bougies M5)
    return s.rolling('30D').median()

print('⌚ Calcul ATR médiane 30j...')
median_atr_series = compute_median_atr_30d(precomp_indicators)
print(f'✅ ATR médiane calculée | min={median_atr_series.min():.1f}'
      f' max={median_atr_series.max():.1f} pts')
```

⌚ Calcul ATR médiane 30j...

✅ ATR médiane calculée | min=9.5 max=26.9 pts

In [31]:

```

# =====
# CELLULE 8 – Backtest Tick principal (cached, NAS100 complet)
# =====
# Implémente :
# §1 Fenêtre temporelle stricte (PREOPEN/OPEN/MID/END + seuils)
# §2 Filtre spread tick + slippage injecté explicitement
# §3 SL dynamique + delayed SL + cap 60 pts
# §5 Trailing post-TP1 (0.4×ATR) et post-TP2 (0.25×range)
# §8 Sizing dynamique (lot variable par trade)
# §9 Circuit breakers : daily stop, consec SL, freq
# =====

def _calc_tp_sl_local(direction, entry, atr, swing_high,
                      swing_low, m, max_risk=60.0,
                      swing_buf=0.30) -> dict:
    if direction == 'BUY':
        sl = min(round(swing_low - atr * swing_buf, 2),
                  round(entry - atr * m['sl_mult'], 2))
        sl = min(sl, round(entry - atr * max(m['sl_mult'], 1.5), 2))
        risk = abs(entry - sl)
        if risk > max_risk:
            sl = round(entry - max_risk, 2); risk = max_risk
        return {'sl': sl,
                'tp1': round(entry + risk * m['tp1_mult'], 2),
                'tp2': round(entry + risk * m['tp2_mult'], 2),
                'tp3': round(entry + risk * m['tp3_mult'], 2)}
    else:
        sl = max(round(swing_high + atr * swing_buf, 2),
                  round(entry + atr * m['sl_mult'], 2))
        sl = max(sl, round(entry + atr * max(m['sl_mult'], 1.5), 2))
        risk = abs(sl - entry)
        if risk > max_risk:
            sl = round(entry + max_risk, 2); risk = max_risk
        return {'sl': sl,
                'tp1': round(entry - risk * m['tp1_mult'], 2),
                'tp2': round(entry - risk * m['tp2_mult'], 2),
                'tp3': round(entry - risk * m['tp3_mult'], 2)}

def run_backtest_nas100(precomp: OrderedDict,
                        tick_cache: dict,
                        median_atr: pd.Series,
                        mode: str = TRADING_MODE,
                        score_min_override: float = None,
                        sl_mult_override: float = None,
                        label: str = 'NAS100 Tick') -> tuple:
    """
    Moteur de backtest tick NAS100 – cahier des charges complet.
    Retourne (df_trades, filter_stats).
    """
    m = copy.deepcopy(MODES_NAS100[mode])
    if score_min_override is not None: m['score_min'] =
score_min_override
    if sl_mult_override is not None: m['sl_mult'] = sl_mult_override

    FORCE_SLIPPAGE = SLIPPAGE_PTS # injecté explicitement, pas lu
    depuis global kernel

```



```

trades_log      = []
daily_pnl       = 0.0
trades_today    = 0
consec_sl       = 0
last_sl_time    = pd.Timestamp('1970-01-01')
last_signal_ts   = pd.Timestamp('1970-01-01')
current_date    = None
daily_stopped   = False

n_no_session = n_spread_rej = n_score_low = 0
n_no_setup   = n_vwap_flat = n_sd_cap = 0
n_daily_stop = n_cooldown  = n_consec_sl = 0
n_freq_limit = 0

for ts, ind in tqdm(precomp.items(),
                    desc=f'🚀 {label}', unit='bougies', ncols=82):
    day = ts.date()
    if day != current_date:
        current_date = day
        daily_pnl     = 0.0
        trades_today  = 0
        consec_sl     = 0
        daily_stopped = False

    # — Circuit breakers journaliers —————
    if daily_stopped:
        n_daily_stop += 1; continue
    if daily_pnl < DAILY_MAX_LOSS:
        daily_stopped = True; n_daily_stop += 1; continue
    if consec_sl >= MAX_CONSECUTIVE_SL:
        n_consec_sl += 1; continue
    if trades_today >= MAX_TRADES_PER_DAY:
        n_freq_limit += 1; continue

    # — Filtre horaire strict NAS100 (§1.3) —————
    h = ts.hour
    if h < SESSION_PREOPEN_H or h >= SESSION_END_H:
        n_no_session += 1; continue

    # — Cooldown dynamique post-SL (§9.2) —————
    cooldown_eff = m['cooldown'] * (COOLDOWN_SL_MULT ** consec_sl)
    if (ts - last_signal_ts).total_seconds() < cooldown_eff:
        n_cooldown += 1; continue

    # — Direction et score —————
    direction, reject = determine_direction(ind)
    if direction is None:
        if reject == 'vwap_flat': n_vwap_flat += 1
        elif reject == 'sd_cap': n_sd_cap += 1
        else: n_no_setup += 1
        continue

    score, _ = compute_score(ind, direction)

    # — Seuil dynamique par heure (§1.4) —————
    threshold = m['score_min']

```

```

if SESSION_PREOPEN_H <= h < SESSION_OPEN_H: threshold += 15
elif SESSION_MID_H <= h < SESSION_END_H: threshold += 10
if (ts - last_sl_time).total_seconds() < 300: threshold += 20

if score < threshold:
    n_score_low += 1; continue

if ts not in tick_cache:
    continue

# — Niveaux TP/SL et sizing dynamique —————
entry_signal = ind['price']
atr          = ind['atr']
levels       = _calc_tp_sl_local(
    direction, entry_signal, atr,
    ind['swing_high'], ind['swing_low'], m,
    max_risk=MAX_RISK_PTS, swing_buf=SWING_BUFFER_MULT
)

med_atr = median_atr.get(ts, atr)
lot      = compute_lot_dynamic(score, atr, h, med_atr)
pnl_pt   = round(lot * CONTRACT_MULTIPLIER_NAS100, 4)

bids, asks = tick_cache[ts]

# — Simulateur tick —————
positions = simulate_trade_ticks_fast(
    direction, entry_signal, levels, ts,
    bids=bids, asks=asks, atr=atr,
    slippage=FORCE_SLIPPAGE
)

# Spread rejeté
if not positions:
    n_spread_rej += 1; continue

# — Rescale PnL selon lot dynamique —————
for p in positions:
    if p['pnl'] != 0.0:
        p['pnl'] = round(p['pnl'] * (pnl_pt / PNL_PER_POINT), 2)

seq_pnl = sum(p['pnl'] for p in positions)
daily_pnl = round(daily_pnl + seq_pnl, 2)
trades_today += 1
last_signal_ts = ts

seq_has_sl = any(p['result'] == 'SL' for p in positions)
if seq_has_sl:
    consec_sl += 1
    last_sl_time = ts
else:
    consec_sl = 0

entry_real = positions[0].get('entry_real', entry_signal)
slippage    = positions[0].get('slippage', 0.0)

for p in positions:

```

```

trades_log.append({
    'open_time':    ts,
    'day':          day,
    'direction':    direction,
    'entry_signal': entry_signal,
    'entry_real':   entry_real,
    'slippage':     slippage,
    'atr':          atr,
    'lot':          lot,
    'pnl_per_pt':   pnl_pt,
    'score':        score,
    'threshold':    threshold,
    'sl':           levels['sl'],
    'tp1':          levels['tp1'],
    'tp2':          levels['tp2'],
    'tp3':          levels['tp3'],
    'position':     p['id'],
    'result':       p['result'],
    'exit_px':      p['exit_px'],
    'pnl':          p['pnl'],
    'mfe':          p.get('mfe', 0.0),
    'mae':          p.get('mae', 0.0),
    'daily_pnl':    daily_pnl,
    'hour':         h,
    'vwap_slope':   ind['vwap_slope'],
    'dist_sd':      ind['dist_vwap_sd'],
    'pattern':      ind['pattern'],
    'vol_ratio':    ind['vol_ratio'],
    'consec_sl':    consec_sl,
})

filter_stats = {
    'no_session':  n_no_session,
    'vwap_flat':   n_vwap_flat,
    'sd_cap':      n_sd_cap,
    'no_setup':    n_no_setup,
    'score_low':   n_score_low,
    'spread_rej':  n_spread_rej,
    'cooldown':    n_cooldown,
    'consec_sl':   n_consec_sl,
    'daily_stop':  n_daily_stop,
    'freq_limit':  n_freq_limit,
}
return pd.DataFrame(trades_log), filter_stats

print('✅ Moteur de backtest NAS100 (tick, cahier des charges complet)
défini')

```

✅ Moteur de backtest NAS100 (tick, cahier des charges complet) défini

In [32]:

```

# =====
# CELLULE 9 – Simulateur M1 wrapper + Run M1 (référence)
# =====

def run_backtest_m1(precomp: OrderedDict,
                    df_m1: pd.DataFrame,
                    median_atr: pd.Series,
                    mode: str = TRADING_MODE) -> tuple:
    """
    Backtest M1 NAS100 (référence approximative).
    Même logique de filtres que le tick, simulateur M1 à la place des
    ticks.
    """
    m = copy.deepcopy(MODES_NAS100[mode])
    trades_log      = []
    daily_pnl       = 0.0
    trades_today    = 0
    consec_sl       = 0
    last_sl_time    = pd.Timestamp('1970-01-01')
    last_signal_ts   = pd.Timestamp('1970-01-01')
    current_date    = None
    daily_stopped   = False
    fs = {k: 0 for k in ['no_session', 'vwap_flat', 'sd_cap',
                        'no_setup', 'score_low', 'cooldown',
                        'consec_sl', 'daily_stop', 'freq_limit']}

    for ts, ind in tqdm(precomp.items(),
                        desc='🚀 M1 (référence)', unit='bougies',
ncols=82):
        day = ts.date()
        if day != current_date:
            current_date = day; daily_pnl = 0.0
            trades_today = 0; consec_sl = 0; daily_stopped = False

        if daily_stopped:
            fs['daily_stop'] += 1; continue
        if daily_pnl < DAILY_MAX_LOSS:
            daily_stopped = True; fs['daily_stop'] += 1; continue
        if consec_sl >= MAX_CONSECUTIVE_SL: fs['consec_sl'] += 1;
        continue
        if trades_today >= MAX_TRADES_PER_DAY: fs['freq_limit'] += 1;
        continue

        h = ts.hour
        if h < SESSION_PREOPEN_H or h >= SESSION_END_H:
            fs['no_session'] += 1; continue

        cooldown_eff = m['cooldown'] * (COOLDOWN_SL_MULT ** consec_sl)
        if (ts - last_signal_ts).total_seconds() < cooldown_eff:
            fs['cooldown'] += 1; continue

        direction, reject = determine_direction(ind)
        if direction is None:
            fs['vwap_flat' if reject == 'vwap_flat' else
              'sd_cap' if reject == 'sd_cap' else 'no_setup'] += 1
            continue

        score, _ = compute_score(ind, direction)

```

```

threshold = m['score_min']
if SESSION_PREOPEN_H <= h < SESSION_OPEN_H: threshold += 15
elif SESSION_MID_H <= h < SESSION_END_H: threshold += 10
if (ts - last_sl_time).total_seconds() < 300: threshold += 20
if score < threshold:
    fs['score_low'] += 1; continue

entry_signal = ind['price']; atr = ind['atr']
levels = _calc_tp_sl_local(
    direction, entry_signal, atr,
    ind['swing_high'], ind['swing_low'], m,
    max_risk=MAX_RISK_PTS, swing_buf=SWING_BUFFER_MULT
)
med_atr = median_atr.get(ts, atr)
lot = compute_lot_dynamic(score, atr, h, med_atr)
pnl_pt = round(lot * CONTRACT_MULTIPLIER_NAS100, 4)

positions = simulate_trade_m1(
    direction, entry_signal, levels, ts, df_m1, atr=atr
)
if not positions:
    continue

for p in positions:
    if p['pnl'] != 0.0:
        p['pnl'] = round(p['pnl'] * (pnl_pt / PNL_PER_POINT), 2)

seq_pnl = sum(p['pnl'] for p in positions)
daily_pnl = round(daily_pnl + seq_pnl, 2)
trades_today += 1; last_signal_ts = ts

if any(p['result'] == 'SL' for p in positions):
    consec_sl += 1; last_sl_time = ts
else:
    consec_sl = 0

for p in positions:
    trades_log.append({
        'open_time': ts, 'day': day, 'direction': direction,
        'entry_signal': entry_signal, 'atr': atr, 'lot': lot,
        'pnl_per_pt': pnl_pt, 'score': score, 'threshold':
threshold,
        'sl': levels['sl'], 'tp1': levels['tp1'],
        'tp2': levels['tp2'], 'tp3': levels['tp3'],
        'position': p['id'], 'result': p['result'],
        'exit_px': p['exit_px'], 'pnl': p['pnl'],
        'daily_pnl': daily_pnl, 'hour': h,
        'vol_ratio': ind['vol_ratio'], 'pattern':
ind['pattern'],
    })

return pd.DataFrame(trades_log), fs

```

— Exécution

```

print('🚀 Lancement backtest M1 (référence)...')
t0 = time.time()

```

```

df_trades_m1, fs_m1 = run_backtest_m1(
    precomp_indicators, df_m1, median_atr_series, TRADING_MODE
)
print(f'✅ M1 terminé en {time.time()-t0:.1f}s | '
      f'{len(df_trades_m1)//3 if not df_trades_m1.empty else 0}
séquences\n')

print('🚀 Lancement backtest Tick NAS100...')
t0 = time.time()
df_trades_tick, fs_tick = run_backtest_nas100(
    precomp_indicators, TICK_CACHE, median_atr_series, TRADING_MODE
)
print(f'✅ Tick terminé en {time.time()-t0:.1f}s | '
      f'{len(df_trades_tick)//3 if not df_trades_tick.empty else 0}
séquences')

```

🚀 Lancement backtest M1 (référence)...

🚀 M1 (référence): 0% | 0/70690
[00:00<?, ?bougies/s]

✅ M1 terminé en 0.3s | 33 séquences

🚀 Lancement backtest Tick NAS100...

🚀 NAS100 Tick: 0% | 0/70690
[00:00<?, ?bougies/s]

✅ Tick terminé en 0.9s | 33 séquences

In [33]:

```

# =====
# CELLULE 10 – Statistiques & Comparaison M1 vs Ticks NAS100
# =====

def compute_stats(df: pd.DataFrame, fs: dict, label: str) -> dict:
    if df.empty:
        print(f'⚠ [{label}] Aucun trade.')
        return {}

    seq = df.groupby('open_time')['pnl'].sum()
    net = round(df['pnl'].sum(), 2)
    gw = df[df['pnl'] > 0]['pnl'].sum()
    gl = abs(df[df['pnl'] < 0]['pnl'].sum())
    pf = round(gw / gl, 2) if gl > 0 else float('inf')
    n_seq = len(seq)
    wr = round((seq > 0).sum() / n_seq * 100, 1) if n_seq > 0 else 0

    equity = seq.cumsum()
    dd = round((equity - equity.cummax()).min(), 2)

    # Calmar ratio (annualisé approx.)
    n_days = max((df['open_time'].max() - df['open_time'].min()).days,
1)
    annual = net * 365 / n_days
    calmar = round(annual / abs(dd), 2) if dd < 0 else float('inf')

    print(f'\n{"="*58}')
    print(f' [{label}] – MODE : {TRADING_MODE.upper()}')
    print(f' Période : {df["open_time"].min().date()} '
          f'→ {df["open_time"].max().date()}')
    print(f' Séquences : {n_seq}')
    print(f' Net Profit : {net:+.2f} $')
    print(f' Profit Factor: {pf}')
    print(f' Max Drawdown : {dd:.2f} $')
    print(f' Calmar Ratio : {calmar}')
    print(f' Winrate (séq): {wr} %')
    print(f' Meilleure : {seq.max():+.2f} $')
    print(f' Pire : {seq.min():+.2f} $')
    print(f'\n Résultats par palier :')
    for tp in ['TP1', 'TP2', 'TP3', 'SL', 'BE', 'OPEN_END']:
        n = (df['result'] == tp).sum()
        pn = df[df['result'] == tp]['pnl'].sum()
        if n > 0:
            print(f' {tp:8s} : {n:4d} fois | PnL cumulé : {pn:+.2f}
    $')

    if 'slippage' in df.columns and df['slippage'].sum() > 0:
        avg_slip = df.groupby('open_time')
['slippage'].first().mean()
        spread_rej_n = fs.get('spread_rej', 0)
        print(f'\n Friction (Ticks) :')
        print(f' Slippage moyen : {avg_slip:.4f} pts')
        print(f' Spread rejets : {spread_rej_n} signaux')

    if 'lot' in df.columns:
        lot_mean = df.groupby('open_time')['lot'].first().mean()
        print(f'\n Sizing dynamique :')

```

```

print(f'    Lot moyen      : {lot_mean:.3f}')

print(f'{"="*58}')
```

return {'net': net, 'pf': pf, 'dd': dd, 'wr': wr, 'calmar': calmar,
 'n_seq': n_seq, 'equity': equity}

```

print('=== STATISTIQUES M1 (référence) ===')
stats_m1 = compute_stats(df_trades_m1, fs_m1, 'M1')

print('\n=== STATISTIQUES TICK NAS100 ===')
stats_tick = compute_stats(df_trades_tick, fs_tick, 'Tick')

# — Tableau comparatif —————
if stats_m1 and stats_tick:
    print(f'\n{"="*65}')
```

📊 COMPARAISON : M1 (approx.) vs Ticks (réel) – NAS100'

```

    print(f'{"="*65}')
```

metrics = [
 ('Net Profit (\$)', stats_m1['net'], stats_tick['net'],
'{:.2f}'),
 ('Profit Factor', stats_m1['pf'], stats_tick['pf'],
'{:.2f}'),
 ('Max Drawdown (\$)', stats_m1['dd'], stats_tick['dd'],
'{:.2f}'),
 ('Winrate %', stats_m1['wr'], stats_tick['wr'],
'{:.1f}'),
 ('Calmar Ratio', stats_m1['calmar'], stats_tick['calmar'],
'{:.2f}'),
 ('Nb séquences', stats_m1['n_seq'], stats_tick['n_seq'],
'{:.0f}'),
]

```

    print(f' {"Métrique":<24} {"M1 (approx)":>13} {"Ticks (réel)":>13}
{"Écart":>9}')
```

```

    print(f' {"-"*24} {"-"*13} {"-"*13} {"-"*9}')
```

```

    for name, v_m1, v_tick, fmt in metrics:
        diff = v_tick - v_m1
        print(f' {"name":<24} {fmt.format(v_m1):>13} '
              f'{fmt.format(v_tick):>13} {"+" if diff>0 else ""}
{diff:>+.2f}')
```

```

    print(f'{"="*65}')
```

=== STATISTIQUES M1 (référence) ===

[M1] – MODE : OPTIMALE
Période : 2025-05-12 → 2026-04-17
Séquences : 33
Net Profit : -264.47 \$
Profit Factor: 0.82
Max Drawdown : -448.17 \$
Calmar Ratio : -0.64
Winrate (séq): 57.6 %
Meilleure : +91.73 \$
Pire : -108.00 \$

Résultats par palier :

TP1 : 19 fois | PnL cumulé : +827.26 \$
TP2 : 5 fois | PnL cumulé : +418.71 \$
SL : 75 fois | PnL cumulé : -1510.44 \$

Sizing dynamique :
Lot moyen : 0.084

=== STATISTIQUES TICK NAS100 ===

[Tick] – MODE : OPTIMALE
Période : 2025-05-12 → 2026-04-17
Séquences : 33
Net Profit : -733.31 \$
Profit Factor: 0.57
Max Drawdown : -727.13 \$
Calmar Ratio : -1.09
Winrate (séq): 45.5 %
Meilleure : +78.30 \$
Pire : -108.00 \$

Résultats par palier :
TP1 : 15 fois | PnL cumulé : +660.11 \$
TP2 : 4 fois | PnL cumulé : +300.80 \$
SL : 80 fois | PnL cumulé : -1694.22 \$

Friction (Ticks) :
Slippage moyen : 0.8788 pts
Spread rejets : 0 signaux

Sizing dynamique :
Lot moyen : 0.084

📊 COMPARAISON : M1 (approx.) vs Ticks (réel) – NAS100

Métrique	M1 (approx)	Ticks (réel)	Écart
Net Profit (\$)	-264.47	-733.31	-468.84
Profit Factor	0.82	0.57	-0.25
Max Drawdown (\$)	-448.17	-727.13	-278.96
Winrate %	57.6	45.5	-12.10
Calmar Ratio	-0.64	-1.09	-0.45
Nb séquences	33	33	+0.00

In [34]:

```

# =====
# CELLULE 11 – Graphiques analytiques : Equity + Journalier + Résultats
# =====

if not df_trades_tick.empty:
    fig = plt.figure(figsize=(24, 18))
    fig.suptitle(
        f'NAS100 Bot – Backtest Tick-by-Tick {TRADING_MODE.upper()} | M1
vs Ticks',
        fontsize=14, fontweight='bold', color='white', y=0.99
    )

    # — (1) Equity curves superposées —————
    ax1 = fig.add_subplot(3, 2, (1, 2))
    eq_tick = stats_tick['equity']
    ax1.fill_between(eq_tick.index, eq_tick.values, alpha=0.15,
color='#00ccff')
    ax1.plot(eq_tick.index, eq_tick.values, lw=2.0, color='#00ccff',
        label=f'Tick Net: {eq_tick.iloc[-1]:+.0f}$')
    if stats_m1:
        eq_m1 = stats_m1['equity']
        ax1.plot(eq_m1.index, eq_m1.values, lw=1.5, color='#ff8800',
            linestyle='--', alpha=0.8,
            label=f'M1 Net: {eq_m1.iloc[-1]:+.0f}$')
    ax1.axhline(0, color='white', lw=0.5, linestyle='--', alpha=0.4)
    ax1.set_title('📊 Equity Curves – M1 (approx.) vs Ticks (réel) –
NAS100',
        fontsize=12, color='#00d4ff')
    ax1.set_ylabel('PnL cumulé ($)', color='white')
    ax1.tick_params(colors='white'); ax1.grid(alpha=0.15)
    ax1.legend(fontsize=10)

    # — (2) PnL journalier —————
    ax2 = fig.add_subplot(3, 2, 3)
    daily = df_trades_tick.groupby('day')['pnl'].sum()
    bar_c = ['#00ccff' if v >= 0 else '#ff4466' for v in daily.values]
    ax2.bar(range(len(daily)), daily.values, color=bar_c, alpha=0.85)
    ax2.axhline(0, color='white', lw=0.5)
    ax2.axhline(DAILY_MAX_LOSS, color='#ff8800', lw=1.2, linestyle='--',
        label=f'Daily Stop {DAILY_MAX_LOSS}$')
    ax2.set_title('📊 PnL Journalier (Ticks)', fontsize=11,
color='#00d4ff')
    ax2.set_ylabel('PnL ($)', color='white')
    ax2.tick_params(colors='white'); ax2.grid(alpha=0.15, axis='y')
    ax2.legend(fontsize=8)

    # — (3) Répartition résultats —————
    ax3 = fig.add_subplot(3, 2, 4)
    rc = df_trades_tick['result'].value_counts()
    pal = {'TP1': '#00ccff', 'TP2': '#0099cc', 'TP3': '#006699',
        'SL': '#ff4466', 'BE': '#ffdd44', 'OPEN_END': '#888888'}
    colors_pie = [pal.get(r, 'aaaaaa') for r in rc.index]
    ax3.pie(rc.values, labels=rc.index, autopct='%1.1f%%',
        colors=colors_pie,
        textprops={'color': 'white', 'fontsize': 9})
    ax3.set_title('📊 Résultats Tick (Positions)', fontsize=11,
color='#00d4ff')

```

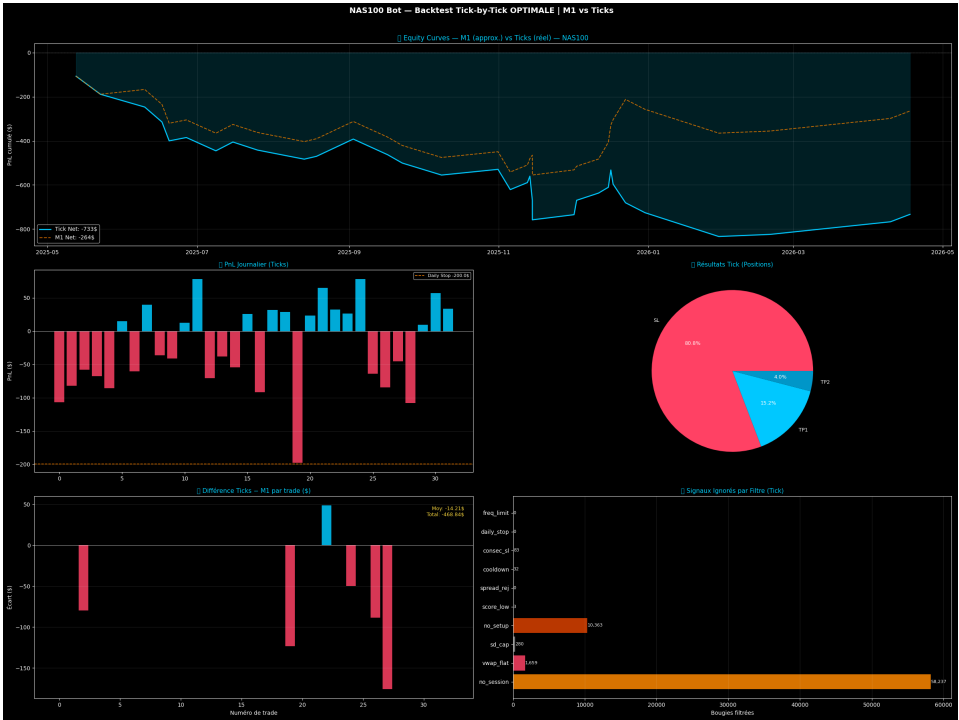
```

# — (4) Différence M1 vs Tick par trade —————
ax4 = fig.add_subplot(3, 2, 5)
if stats_m1 and not df_trades_m1.empty:
    seq_m1 = df_trades_m1.groupby('open_time')['pn1'].sum()
    seq_tick = df_trades_tick.groupby('open_time')['pn1'].sum()
    comp_df = pd.concat([seq_m1, seq_tick],
                        axis=1, keys=['m1', 'tick']).dropna()
    diff = comp_df['tick'] - comp_df['m1']
    diff_c = ['#00ccff' if v >= 0 else '#ff4466' for v in
diff.values]
    ax4.bar(range(len(diff)), diff.values, color=diff_c, alpha=0.85)
    ax4.axhline(0, color='white', lw=0.5)
    ax4.set_title('📊 Différence Ticks - M1 par trade ($)',
                  fontsize=11, color='#00d4ff')
    ax4.set_xlabel('Numéro de trade', color='white')
    ax4.set_ylabel('Écart ($)', color='white')
    ax4.tick_params(colors='white'); ax4.grid(alpha=0.15, axis='y')
    ax4.text(0.98, 0.95,
             f'Moy: {diff.mean():+.2f}$\nTotal: {diff.sum():+.2f}$',
             transform=ax4.transAxes, va='top', ha='right',
             color='#ffdd44', fontsize=9)

# — (5) Filtres —————
ax5 = fig.add_subplot(3, 2, 6)
lf = list(fs_tick.keys())
vf = list(fs_tick.values())
cf = ['#ff8800', '#ff4466', '#888888', '#dd4400',
      '#ff6600', '#cc2200', '#996600', '#774400', '#552200',
'#330000']
bars5 = ax5.barh(lf, vf, color=cf[:len(lf)], alpha=0.85)
ax5.set_title('🚫 Signaux Ignorés par Filtre (Tick)',
              fontsize=11, color='#00d4ff')
ax5.set_xlabel('Bougies filtrées', color='white')
ax5.tick_params(colors='white'); ax5.grid(alpha=0.15, axis='x')
for bar, val in zip(bars5, vf):
    ax5.text(bar.get_width() + 10, bar.get_y() + bar.get_height()/2,
             f'{val:,.}', va='center', color='white', fontsize=8)

plt.tight_layout(rect=[0, 0, 1, 0.98])
plt.savefig('nas100_backtest_results.png', dpi=150,
bbox_inches='tight',
          facecolor='#0d0d0d', edgecolor='none')
plt.show()
print('✅ Graphique sauvegardé → nas100_backtest_results.png')

```



✓ Graphique sauvegardé → nas100_backtest_results.png

In [35]:

```

# =====
# CELLULE 12 – Scatter Plot MFE / MAE (module analytique)
# =====
# MFE = Maximum Favorable Excursion (gain max latent en pts)
# MAE = Maximum Adverse Excursion (perte max latente en pts, négatif)
# Permet d'identifier :
#   - TP trop serrés (MFE >> TP atteint)
#   - SL trop larges (MAE profond sur trades SL)
#   - Qualité des entrées (MAE faible → entrée précise)
# =====

if not df_trades_tick.empty and 'mfe' in df_trades_tick.columns:
    # Regrouper par séquence (P1 = représentant)
    df_p1 = df_trades_tick[df_trades_tick['position'] == 'P1'].copy()
    df_p1['seq_pn1'] = df_trades_tick.groupby('open_time')
    ['pn1'].sum().reindex(
        df_p1['open_time'].values).values

    fig2, axes2 = plt.subplots(1, 2, figsize=(20, 7))
    fig2.suptitle('NAS100 – Analyse MFE / MAE (Qualité d\'Entrée)',
        fontsize=13, fontweight='bold', color='white')

    # — Scatter MFE vs MAE coloré par résultat —————
    ax_s = axes2[0]
    pal_res = {'TP1': '#00ccff', 'TP2': '#0099cc', 'TP3': '#006699',
        'SL': '#ff4466', 'BE': '#ffdd44', 'OPEN_END': '#888888'}

    for res in df_p1['result'].unique():
        sub = df_p1[df_p1['result'] == res]
        ax_s.scatter(sub['mae'], sub['mfe'],
            c=pal_res.get(res, 'aaaaaa'), label=res,
            alpha=0.75, s=55, edgecolors='none')

    ax_s.axhline(0, color='white', lw=0.5, alpha=0.4)
    ax_s.axvline(0, color='white', lw=0.5, alpha=0.4)
    ax_s.set_xlabel('MAE (pts) – excursion adverse max', color='white')
    ax_s.set_ylabel('MFE (pts) – excursion favorable max',
        color='white')
    ax_s.set_title('Scatter MFE vs MAE (par position P1)',
        color='#00d4ff')
    ax_s.tick_params(colors='white'); ax_s.grid(alpha=0.15)
    ax_s.legend(fontsize=8)

    # Ligne idéale : MFE = -MAE (entrée parfaite, aucune adverse)
    lim = max(abs(df_p1['mae'].min()), df_p1['mfe'].max())
    ax_s.plot([-lim, 0], [0, lim], color='aaaaaa', lw=0.8,
        linestyle=':', alpha=0.5, label='MFE=-MAE (idéal)')

    # — Distribution MAE par résultat (boîte à moustaches) —
    ax_b = axes2[1]
    result_order = ['TP1', 'TP2', 'TP3', 'SL', 'BE']
    data_box = [df_p1[df_p1['result'] == r]['mae'].dropna().values
        for r in result_order if (df_p1['result'] == r).any()]
    labels_box = [r for r in result_order if (df_p1['result'] ==
        r).any()]

    bp = ax_b.boxplot(data_box, labels=labels_box, patch_artist=True,

```

```

        medianprops=dict(color='white', lw=2))
for patch, label in zip(bp['boxes'], labels_box):
    patch.set_facecolor(pal_res.get(label, '#aaaaaa'))
    patch.set_alpha(0.75)
for whisker in bp['whiskers']:
    whisker.set(color='white', alpha=0.5)
for cap in bp['caps']:
    cap.set(color='white', alpha=0.5)
for flier in bp['fliers']:
    flier.set(marker='.', color='#888888', alpha=0.5)

ax_b.axhline(0, color='white', lw=0.5, linestyle='--', alpha=0.4)
ax_b.set_xlabel('Résultat P1', color='white')
ax_b.set_ylabel('MAE (pts)', color='white')
ax_b.set_title('Distribution MAE par Résultat P1', color='#00d4ff')
ax_b.tick_params(colors='white'); ax_b.grid(alpha=0.15, axis='y')

# Statistiques MAE/MFE
print(f'\n📊 Statistiques MFE/MAE (positions P1) :')
print(f'    MFE moyen (tous)      : {df_p1["mfe"].mean():+.1f} pts')
print(f'    MAE moyen (tous)       : {df_p1["mae"].mean():+.1f} pts')
print(f'    MAE moyen (SL)         : '
      f'{df_p1[df_p1["result"]=="SL"]["mae"].mean():+.1f} pts')
print(f'    MAE moyen (TP1)        : '
      f'{df_p1[df_p1["result"]=="TP1"]["mae"].mean():+.1f} pts')
mfe_above_tp1 = (df_p1['mfe'] > (
    (df_p1['tp1'] - df_p1['entry_real']) if 'entry_real' in df_p1
else 0
)).sum()
print(f'    Trades MFE > TP1 mais SL : '
      f'{len(df_p1[(df_p1["mfe"] > 30) & (df_p1["result"] ==
"SL")])})')

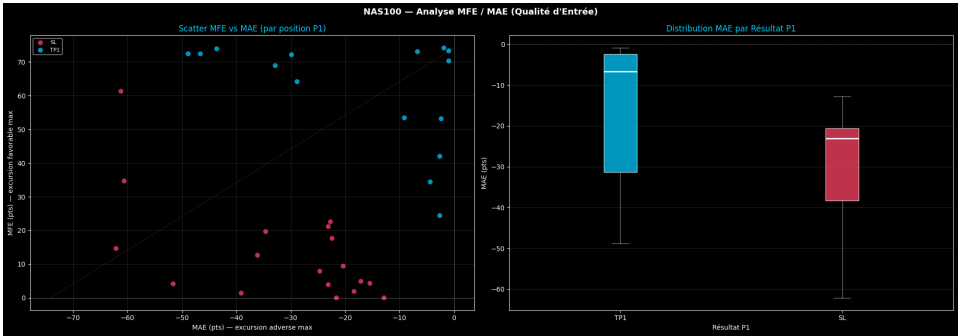
plt.tight_layout()
plt.savefig('nas100_mfe_mae.png', dpi=130, bbox_inches='tight',
           facecolor='#0d0d0d')
plt.show()
print('✅ MFE/MAE sauvegardé → nas100_mfe_mae.png')
else:
    print('⚠️ Colonnes MFE/MAE non disponibles ou DataFrame vide.')

```

```

📊 Statistiques MFE/MAE (positions P1) :
    MFE moyen (tous)      : +35.3 pts
    MAE moyen (tous)       : -25.1 pts
    MAE moyen (SL)         : -31.5 pts
    MAE moyen (TP1)        : -17.5 pts
    Trades MFE > TP1 mais SL : 2

```



✓ MFE/MAE sauvegardé → nas100_mfe_mae.png

In [36]:

```

# =====
# CELLULE 13 – Matrice de sensibilité parallèle (Score Min × SL Mult)
# =====
# ThreadPoolExecutor sur tous les cœurs disponibles.
# Chaque worker reçoit une copie locale des données (pas de partage).
# =====

N_CORES = multiprocessing.cpu_count()
print(f'🖥️ CPU : {N_CORES} cœurs disponibles')

def _sensitivity_worker_nas100(args: tuple) -> dict:
    """
    Worker autonome NAS100 – toutes les constantes passées par valeur.
    Pas de lecture de variables globales (picklable, thread-safe).
    """
    (precomp_items, tick_cache_local, mode,
     sm, slm, modes_cfg, pnl_pt,
     slip, max_risk, swing_buf,
     vwap_slope_ratio, sd_cap_v,
     ses_preopen, ses_open, ses_mid, ses_end,
     max_trades, max_consec, cooldown_sl_mult,
     daily_max, atr_trail, tp2_lock,
     delayed_sl_mult, max_spread,
     contract_mult) = args

    import copy as _copy
    import pandas as _pd
    import numpy as _np
    from collections import OrderedDict

    precomp = OrderedDict(precomp_items)
    m = _copy.deepcopy(modes_cfg[mode])
    m['score_min'] = sm
    m['sl_mult'] = slm

    def _tp_sl(direction, entry, atr, sh, sl_v, mm):
        if direction == 'BUY':
            sl = min(round(sl_v - atr*swing_buf, 2),
                     round(entry - atr*mm['sl_mult'], 2))
            sl = min(sl, round(entry - atr*max(mm['sl_mult'], 1.5), 2))
            risk = abs(entry - sl)
            if risk > max_risk: sl = round(entry - max_risk, 2); risk =
max_risk
            return {'sl': sl,
                    'tp1': round(entry + risk*mm['tp1_mult'], 2),
                    'tp2': round(entry + risk*mm['tp2_mult'], 2),
                    'tp3': round(entry + risk*mm['tp3_mult'], 2)}
        else:
            sl = max(round(sh + atr*swing_buf, 2),
                     round(entry + atr*mm['sl_mult'], 2))
            sl = max(sl, round(entry + atr*max(mm['sl_mult'], 1.5), 2))
            risk = abs(sl - entry)
            if risk > max_risk: sl = round(entry + max_risk, 2); risk =
max_risk
            return {'sl': sl,
                    'tp1': round(entry - risk*mm['tp1_mult'], 2),

```



```

        'tp2': round(entry - risk*mm['tp2_mult'], 2),
        'tp3': round(entry - risk*mm['tp3_mult'], 2)}

def _simulate(direction, entry_sig, levels, bids, asks):
    spread = asks[0] - bids[0]
    if spread > max_spread:
        return []
    entry = round((asks[0]+slip) if direction=='BUY' else (bids[0]-
slip), 2)
    delta = entry - entry_sig
    sl = round(levels['sl'] + delta, 2)
    tp1 = round(levels['tp1'] + delta, 2)
    tp2 = round(levels['tp2'] + delta, 2)
    tp3 = round(levels['tp3'] + delta, 2)
    pos = [

{'id':'P1','tp':tp1,'sl':sl,'pn1':0.,'result':None,'closed':False},

{'id':'P2','tp':tp2,'sl':sl,'pn1':0.,'result':None,'closed':False},

{'id':'P3','tp':tp3,'sl':sl,'pn1':0.,'result':None,'closed':False},
    ]
    tp1h = tp2h = False
    ep = bids if direction=='BUY' else asks
    for px in ep:
        if all(p['closed'] for p in pos): break
        for p in pos:
            if p['closed']: continue
            th = (px>=p['tp']) if direction=='BUY' else
(px<=p['tp'])
            sh = (px<=p['sl']) if direction=='BUY' else
(px>=p['sl'])
            if th and sh:
                th = abs(p['tp']-entry) < abs(p['sl']-entry); sh =
not th
            if th:
                p['closed']=True; pts=(p['tp']-entry) if
direction=='BUY' else (entry-p['tp'])
                p['result']=p['id'].replace('P','TP');
                p['pn1']=round(pts*pn1_pt, 2)
                if p['id']=='P1' and not tp1h:
                    tp1h=True
                    trail_sl = round((entry + atr_trail*0) if
direction=='BUY' else
                                (entry - atr_trail*0), 2) #
simplified: entry
                for o in pos:
                    if not o['closed']:
                        if direction=='BUY' and entry >
o['sl']: o['sl'] = entry
                        elif direction=='SELL' and entry <
o['sl']: o['sl'] = entry
                    if p['id']=='P2' and not tp2h and tp1h:
                        tp2h=True
                        if direction=='BUY':
                            lock = round(tp1 + tp2_lock*(tp2-tp1), 2)
                        else:

```

```

        lock = round(tp1 - tp2_lock*(tp1-tp2), 2)
    for o in pos:
        if not o['closed']:
            if direction=='BUY' and lock > o['sl']:
o['sl'] = lock
            elif direction=='SELL' and lock <
o['sl']: o['sl'] = lock
            elif sh:
                p['closed']=True; pts=abs(entry-p['sl'])
                p['result']='BE' if abs(p['sl']-entry)<0.01 else
'SL'
                p['pnl'] = 0. if p['result']=='BE' else round(-
pts*pnl_pt, 2)
                lp = ep[-1]
                for p in pos:
                    if not p['closed']:
                        pts=(lp-entry) if direction=='BUY' else (entry-lp)
                        p['result']='OPEN_END'; p['pnl']=round(pts*pnl_pt,2);
p['closed']=True
                return pos

pnl_s=[]; daily_pnl=0.; trades_today=0; consec_sl=0
last_sl_ts=_pd.Timestamp('1970-01-01')
last_sig_ts=_pd.Timestamp('1970-01-01')
cur_date=None; daily_stopped=False

for ts, ind in precomp.items():
    day = ts.date()
    if day != cur_date:
        cur_date=day; daily_pnl=0.; trades_today=0
        consec_sl=0; daily_stopped=False
    if daily_stopped: continue
    if daily_pnl < daily_max: daily_stopped=True; continue
    if consec_sl >= max_consec: continue
    if trades_today >= max_trades: continue
    h = ts.hour
    if h < ses_preopen or h >= ses_end: continue
    cooldown_eff = m['cooldown'] * (cooldown_sl_mult ** consec_sl)
    if (ts-last_sig_ts).total_seconds() < cooldown_eff: continue

    # Direction inline
    if abs(ind['vwap_slope']) < ind['atr']*vwap_slope_ratio:
continue
        dist = ind['dist_vwap_sd']
        if dist > sd_cap_v or dist < -sd_cap_v: continue
        price = ind['price']
        direction = None
        if ind['bounce_bull']: direction='BUY'
        elif ind['bounce_bear']: direction='SELL'
        elif price<ind['vwap_lower1'] and (ind['near_swing_sup'] or
ind['near_poc']) \
            and ind['above_200'] and ind['above_50'] and
ind['vol_ratio']>1.0:
            direction='BUY'
        elif price>ind['vwap_upper1'] and (ind['near_swing_res'] or
ind['near_poc']) \
            and not ind['above_200'] and not ind['above_50'] and

```

```

ind['vol_ratio']>1.0:
    direction='SELL'
    elif price<ind['vwap_lower2'] and ind['above_200'] and
ind['vol_ratio']>1.0:
    direction='BUY'
    elif price>ind['vwap_upper2'] and not ind['above_200'] and
ind['vol_ratio']>1.0:
    direction='SELL'
    if direction is None: continue

# Score inline
score=0; is_buy=(direction=='BUY')
dsd=abs(dist)
if dsd>=2.0: score+=30
elif dsd>=1.0: score+=18
elif dsd>=0.5: score+=8
vs=ind['vwap_slope']
if is_buy and vs>0: score+=5
elif not is_buy and vs<0: score+=5
if is_buy:
    if ind['near_swing_sup']: score+=15
    if ind['near_poc']: score+=10
    if price<ind['va_low']: score+=10
    if ind['above_200']: score+=5
else:
    if ind['near_swing_res']: score+=15
    if ind['near_poc']: score+=10
    if price>ind['va_high']: score+=10
    if not ind['above_200']: score+=5
vr=ind['vol_ratio']
if ind['bounce_bull'] or ind['bounce_bear']: score+=25
elif vr>=2.0: score+=20
elif vr>=1.4: score+=12
elif vr>=1.0: score+=5
if is_buy and ind['ema_cross_bull']: score+=10
elif not is_buy and ind['ema_cross_bear']: score+=10
pat=ind['pattern']
bull_p={'Marteau','Engulfing haussier','Pin Bar haussier'}
bear_p={'Shooting Star','Engulfing baissier','Pin Bar baissier'}
if is_buy and pat in bull_p: score+=15
elif not is_buy and pat in bear_p: score+=15
elif pat=='Doji': score+=3
score=min(score,100)

threshold=m['score_min']
if ses_preopen<=h<ses_open: threshold+=15
elif ses_mid<=h<ses_end: threshold+=10
if (ts-last_sl_ts).total_seconds(<300: threshold+=20
if score<threshold: continue
if ts not in tick_cache_local: continue

lvl = _tp_sl(direction, price, ind['atr'],
            ind['swing_high'], ind['swing_low'], m)
bids, asks = tick_cache_local[ts]
pos = _simulate(direction, price, lvl, bids, asks)
if not pos: continue

```

```

seq = sum(p['pnl'] for p in pos)
daily_pnl=round(daily_pnl+seq,2)
trades_today+=1; last_sig_ts=ts
if any(p['result']=='SL' for p in pos):
    consec_sl+=1; last_sl_ts=ts
else:
    consec_sl=0
pnls.append(seq)

if not pnls:
    return {'score_min': sm, 'sl_mult': slm,
            'net_profit': 0, 'n_seq': 0, 'winrate': 0, 'pf': 0,
'calmar': 0}
arr = _np.array(pnls)
net = round(arr.sum(), 2)
n_seq= len(arr)
wr = round((arr>0).sum()/n_seq*100, 1)
gw = arr[arr>0].sum()
gl = abs(arr[arr<0].sum())
pf = round(gw/gl, 2) if gl>0 else 999
equity = _np.cumsum(arr)
dd = round((_np.minimum.accumulate(equity) -
_np.maximum.accumulate(equity)).min(), 2)
calmar = round((net / abs(dd)), 2) if dd < 0 else 999
return {'score_min': sm, 'sl_mult': slm,
        'net_profit': net, 'n_seq': n_seq,
        'winrate': wr, 'pf': pf, 'calmar': calmar}

def run_sensitivity_parallel(precomp: OrderedDict,
                             tick_cache: dict,
                             mode: str,
                             score_range: list,
                             sl_range: list) -> pd.DataFrame:
    precomp_list = list(precomp.items())
    modes_copy = copy.deepcopy(MODES_NAS100)

    combos = [
        (precomp_list, tick_cache, mode, sm, slm, modes_copy,
PNL_PER_POINT,
        SLIPPAGE_PTS, MAX_RISK_PTS, SWING_BUFFER_MULT,
        VWAP_SLOPE_ATR_RATIO, SD_CAP,
        SESSION_PREOPEN_H, SESSION_OPEN_H, SESSION_MID_H,
SESSION_END_H,
        MAX_TRADES_PER_DAY, MAX_CONSECUTIVE_SL, COOLDOWN_SL_MULT,
        DAILY_MAX_LOSS, ATR_TRAIL_P1, TP2_LOCK_RATIO,
        DELAYED_SL_MULT, MAX_SPREAD_NAS100,
        CONTRACT_MULTIPLIER_NAS100)
        for sm in score_range for slm in sl_range
    ]

    results = []
    n_workers= min(N_CORES, len(combos))
    print(f'🚀 {len(combos)} combinaisons | {n_workers} workers CPU')
    t0 = time.time()

    with ThreadPoolExecutor(max_workers=n_workers) as ex:

```

```

futures = {ex.submit(_sensitivity_worker_nas100, a): (a[3],
a[4])
            for a in combos}
with tqdm(total=len(combos), desc='Sensibilité NAS100',
          unit='combo', ncols=70) as pbar:
    for fut in as_completed(futures):
        try:
            results.append(fut.result())
        except Exception as e:
            sm, slm = futures[fut]
            results.append({'score_min': sm, 'sl_mult': slm,
                           'net_profit': 0, 'n_seq': 0,
                           'winrate': 0, 'pf': 0, 'calmar': 0})
            print(f' ⚠ Erreur sm={sm} slm={slm}: {e}')
        pbar.update(1)

elapsed = time.time() - t0
print(f'\n✅ Terminé en {elapsed:.1f}s
({elapsed/len(combos):.1f}s/combo)')
return pd.DataFrame(results)

# — Lancement —————
print('📊 Analyse de sensibilité NAS100 PARALLÈLE...\n')
score_range = [40, 45, 50, 55, 60, 65, 70, 75]
sl_range     = [1.5, 1.8, 2.0, 2.2, 2.5]

df_sens = run_sensitivity_parallel(
    precomp_indicators, TICK_CACHE, TRADING_MODE,
    score_range, sl_range
)

```

🖥️ CPU : 6 cœurs disponibles

📊 Analyse de sensibilité NAS100 PARALLÈLE...

🚀 40 combinaisons | 6 workers CPU

Sensibilité NAS100: 0% | 0/40 [00:00<?, ?
combo/s]

✅ Terminé en 22.3s (0.6s/combo)

In [37]:

```

# =====
# CELLULE 14 – Heatmaps de sensibilité (Net Profit + Winrate + Calmar)
# =====

pivot_net    = df_sens.pivot(index='sl_mult', columns='score_min',
                              values='net_profit')
pivot_wr     = df_sens.pivot(index='sl_mult', columns='score_min',
                              values='winrate')
pivot_calmar = df_sens.pivot(index='sl_mult', columns='score_min',
                              values='calmar')

fig3, axes3 = plt.subplots(1, 3, figsize=(24, 6))
fig3.suptitle(f'Sensibilité NAS100 Tick – {TRADING_MODE.upper()}',
              fontsize=13, fontweight='bold', color='white')

def _heatmap(ax, pivot, title, fmt, cmap='RdYlGn', vmin=None,
             vmax=None):
    im = ax.imshow(pivot.values, aspect='auto', cmap=cmap,
                   vmin=vmin, vmax=vmax)
    ax.set_xticks(range(len(pivot.columns)))
    ax.set_xticklabels(pivot.columns, color='white')
    ax.set_yticks(range(len(pivot.index)))
    ax.set_yticklabels(pivot.index, color='white')
    ax.set_xlabel('Score Min', color='white')
    ax.set_ylabel('SL Mult', color='white')
    ax.set_title(title, color='#00d4ff')
    for i in range(len(pivot.index)):
        for j in range(len(pivot.columns)):
            v = pivot.values[i, j]
            ax.text(j, i, fmt.format(v), ha='center', va='center',
                   fontsize=8,
                   color='black' if abs(v) < (pivot.values.max()*0.6)
else 'white')
    plt.colorbar(im, ax=ax)

_heatmap(axes3[0], pivot_net, 'Net Profit ($)', '{:.0f}')
_heatmap(axes3[1], pivot_wr, 'Winrate (%)', '{:.0f}%', vmin=30,
vmax=80)
_heatmap(axes3[2], pivot_calmar, 'Calmar Ratio', '{:.1f}',
cmap='RdYlGn')

plt.tight_layout()
plt.savefig('nas100_sensitivity.png', dpi=130, bbox_inches='tight',
          facecolor='#0d0d0d')
plt.show()

# — Résumé meilleure combinaison —————
best_net    = df_sens.loc[df_sens['net_profit'].idxmax()]
best_calmar = df_sens.loc[df_sens['calmar'].idxmax()]

print(f'\n 🏆 Meilleure combinaison (Net Profit) :')
print(f'   score_min={best_net["score_min"]:.0f} | sl_mult=
{best_net["sl_mult"]:.1f}'
      f' → Net: {best_net["net_profit"]:+.2f}$ | WR:
{best_net["winrate"]}%'
      f' | Calmar: {best_net["calmar"]:.2f}')

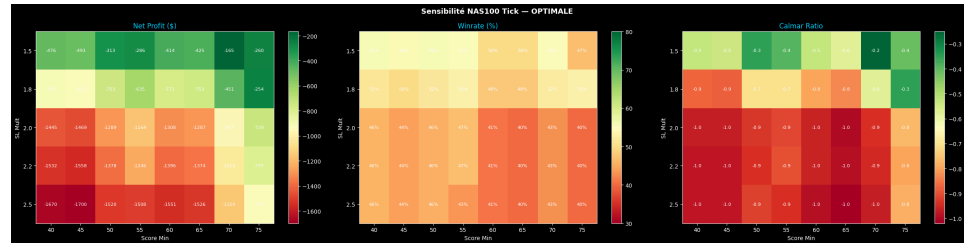
```

```

print(f'\n🏆 Meilleure combinaison (Calmar Ratio) :')
print(f'    score_min={best_calmar["score_min"]:.0f} | sl_mult=
{best_calmar["sl_mult"]:.1f}'
      f' → Net: {best_calmar["net_profit"]:+.2f}$ | WR:
{best_calmar["winrate"]}%
      f' | Calmar: {best_calmar["calmar"]:.2f}')

print(f'\n💻 {N_CORES} cœurs utilisés simultanément')
print('✅ Sensibilité sauvegardée → nas100_sensitivity.png')

```



🏆 Meilleure combinaison (Net Profit) :
 score_min=70 | sl_mult=1.5 → Net: -165.27\$ | WR: 54.5% |
 Calmar: -0.25

🏆 Meilleure combinaison (Calmar Ratio) :
 score_min=70 | sl_mult=1.5 → Net: -165.27\$ | WR: 54.5% |
 Calmar: -0.25

💻 6 cœurs utilisés simultanément
 ✅ Sensibilité sauvegardée → nas100_sensitivity.png

In [38]:

```
# =====
# CELLULE 15 – Notes de référence NAS100
# =====

print("""



---


🚀 NOTES – NAS100 Bot Backtest Tick-by-Tick
(Cahier des charges du 09/05/2026)



---



Architecture :
    M5 → signaux (indicateurs VWAP/ATR, score, filtres)
    Ticks (Parquet LazyFrame) → exécution chirurgicale

Fenêtre temporelle NAS100 (GMT+2) :
    < 14h          : interdit (consolidation nocturne)
    14h-15h        : autorisé, seuil score_min + 15
    15h-16h        : fenêtre optimale, seuil nominal
    16h-18h        : autorisé, seuil score_min + 10
    ≥ 18h          : hard block absolu

Friction simulée :
    BUY  : entrée Ask + 2.0 pts (slippage Open US)
    SELL : entrée Bid - 2.0 pts
    Refus si spread > 1.5 pts
    Trailing post-TP1 : SL → entry ± 0.4×ATR
    Lock post-TP2    : SL → tp1 + 0.25×(tp2-tp1)

Paramètres fixes (cahier des charges §10) :
    CONTRACT_MULTIPLIER = 10 $/pt/lot
    LOT_BASE              = 0.10 → 1.00 $/pt
    MAX_RISK_PTS          = 60 pts → 60 $ max/sous-trade
    DAILY_MAX_LOSS        = -200 $ (3 séq. SL complètes)
    MAX_TRADES/JOUR       = 20
    MAX_CONSEC_SL         = 2 (circuit breaker)
    COOLDOWN_SL_MULT      = 2.0× (décroissance agressive)
    DELAYED_SL            = 1 bougie M5 (vs 2 XAUUSD)
    SWING_BUFFER           = 0.30×ATR (vs 0.20 XAUUSD)
    VWAP_SLOPE_FILTER     = 0.08×ATR (vs 0.10 XAUUSD)

Sorties :
    nas100_backtest_results.png – Equity + Journalier + Filtres
    nas100_mfe_mae.png          – Scatter MFE/MAE
    nas100_sensitivity.png       – Heatmaps Net/WR/Calmar

⚠️ Vérifier CONTRACT_MULTIPLIER_NAS100 (VTMarkets)
avant déploiement live.

""")
```

🚀 NOTES – NAS100 Bot Backtest Tick-by-Tick
(Cahier des charges du 09/05/2026)

Architecture :

M5 → signaux (indicateurs VWAP/ATR, score, filtres)
Ticks (Parquet LazyFrame) → exécution chirurgicale

Fenêtre temporelle NAS100 (GMT+2) :

< 14h : interdit (consolidation nocturne)
14h-15h : autorisé, seuil score_min + 15
15h-16h : fenêtre optimale, seuil nominal
16h-18h : autorisé, seuil score_min + 10
≥ 18h : hard block absolu

Friction simulée :

BUY : entrée Ask + 2.0 pts (slippage Open US)
SELL : entrée Bid - 2.0 pts
Refus si spread > 1.5 pts
Trailing post-TP1 : SL → entry ± 0.4×ATR
Lock post-TP2 : SL → tp1 + 0.25×(tp2-tp1)

Paramètres fixes (cahier des charges \$10) :

CONTRACT_MULTIPLIER = 10 \$/pt/lot
LOT_BASE = 0.10 → 1.00 \$/pt
MAX_RISK_PTS = 60 pts → 60 \$ max/sous-trade
DAILY_MAX_LOSS = -200 \$ (3 séq. SL complètes)
MAX_TRADES/JOUR = 20
MAX_CONSEC_SL = 2 (circuit breaker)
COOLDOWN_SL_MULT = 2.0× (décroissance agressive)
DELAYED_SL = 1 bougie M5 (vs 2 XAUUSD)
SWING_BUFFER = 0.30×ATR (vs 0.20 XAUUSD)
VWAP_SLOPE_FILTER = 0.08×ATR (vs 0.10 XAUUSD)

Sorties :

nas100_backtest_results.png – Equity + Journalier + Filtres
nas100_mfe_mae.png – Scatter MFE/MAE
nas100_sensitivity.png – Heatmaps Net/WR/Calmar

⚠ Vérifier CONTRACT_MULTIPLIER_NAS100 (VTMarkets)
avant déploiement live.
